

Bachelor thesis

Integrating GitLab with Moodle

09.07.2026

Jouve Léonard

HEIG-VD

Contents

1. Introduction	1
1.1. Motivation	1
1.2. Goal	2
2. Structure	2
3. Context	3
3.1. Moodle	3
3.2. GitLab	4
4. State of the art	6
4.1. GitLab Client	6
4.2. Moodle Plugin	6
4.3. Integrations	7
4.3.1. GitHub Classroom	7
4.3.2. RepoBee	8
4.3.3. Classmoji	8
4.4. Academic work	9
4.4.1. Canvas GitLab Integration	9
5. Specifications	10
5.1. Objectives	10
5.2. Deliverables	10
5.3. Expected result	10
6. System architecture	11
7. Features	12
7.1. Teachers	12
7.2. Students	13
7.3. Automation	14
7.4. Other	15
8. Integration Flows	16
9. Development Process	16
9.1. Proof Of Concept	16
10. Solution	18
11. Lessons Learned and Future Work	18

Contents	ii
12. Appendices	19
12.1. Planning	19
12.2. Weekly meeting reports	19
12.3. Source code	19
12.4. Kanban Board	19
Bibliography	20

1. Introduction

In higher education, digital platforms play a central role in courses organization, learning materials distribution, and academic activities management. Learning Management Systems (LMS) such as Moodle are widely adopted to structure the educational experience for both teachers and students. At the same time, in computer science and related fields, source code management platforms like GitLab have become essential tools for distributing assignments, enabling collaboration, and evaluating practical work.

However, these two types of platforms, while complementary, operate independently. This separation leads to fragmented workflows: teachers must manually manage repositories, student groups, and assessment processes on GitLab, while maintaining course-related information on Moodle. For students, this results in scattered resources and information across multiple platforms, which can lead to confusion and reduce overall learning experience.

1.1. Motivation

The lack of integration between LMS platforms and Git repository hosting services introduces several challenges. Teachers often face repetitive and time-consuming tasks such as creating repositories for each student or group, managing access permissions, tracking contributions, and check submission date. These manual processes are not only inefficient but also prone to errors, especially in courses with a large number of students.

Students, on the other hand, must navigate between multiple platforms to access learning materials, enrolment information, starter code, assessment results and feedback. This fragmentation can create confusion and reduce engagement as important informations are not centralized.

Existing solutions partially address these issues but remain limited. Some are proprietary and lack flexibility, while others are tightly coupled to specific platforms such as GitHub. Moreover, they are often not deeply integrated into LMS environments, requiring additional tools or workflows that increase complexity rather than reduce it. This situation highlights the need for an open, flexible, and LMS-native solution that seamlessly integrates Git-based workflows into the educational ecosystem.

1.2. Goal

To address these challenges, this project aims to develop of an extensible integration between Moodle and GitLab. The solution consists of a custom Moodle plugin that communicates with the GitLab API. This integration will enable direct interaction with GitLab features from within the Moodle interface.

The integration will automate workflows involved in programming assignments. This includes the creation and management of repositories, automatic setup of student groups, synchronization of project data, and integration of feedback and grading mechanisms. By embedding these functionalities into Moodle, both teachers and students benefit from a unified platform where all relevant information and actions are accessible.

From an architectural perspective, the solution is designed to be modular and extensible. While initially focused on GitLab, the system will be structured in a way that allows future support for other Git repository hosting services. This ensures long-term adaptability and encourages community contributions.

2. Structure

The remainder of this report is organized as follows:

- **Context** introduces the different platforms and technologies involved in the project.
- **State of the Art** reviews and evaluates existing solutions related to the proposed integration.
- **Specifications** defines the project's objectives, deliverables, requirements, and expected outcomes.
- **System Architecture** describes the architectural design chosen.
- **Features** presents a detailed description of the functionalities provided by the developed integration.
- **Integration Flows** provide graphical representations of the interactions between the different components of the system.
- **Development Process** outlines the main stages of the project's development and implementation process.
- **Solution** details the final delivered solution.
- **Lessons Learned and Future Work** summarizes the project's outcomes, discusses the challenges encountered and the lessons learned, and presents potential improvements and directions for future development.
- **Appendices** contain the supplementary material accompanying this report.

3. Context

This chapter introduces the main platforms and technologies involved in the project. The presented information aims to provide a better understanding of the overall system and the role of each component within it.

3.1. Moodle

Moodle[1] is an open-source Learning Management System (LMS) widely used by universities and educational institutions to organize courses, distribute learning materials, and manage academic activities. Its name stands for Modular Object-Oriented Dynamic Learning Environment. Moodle provides a modular structure based on courses, activities, and plugins, enabling institutions to adapt the platform according to their specific requirements.

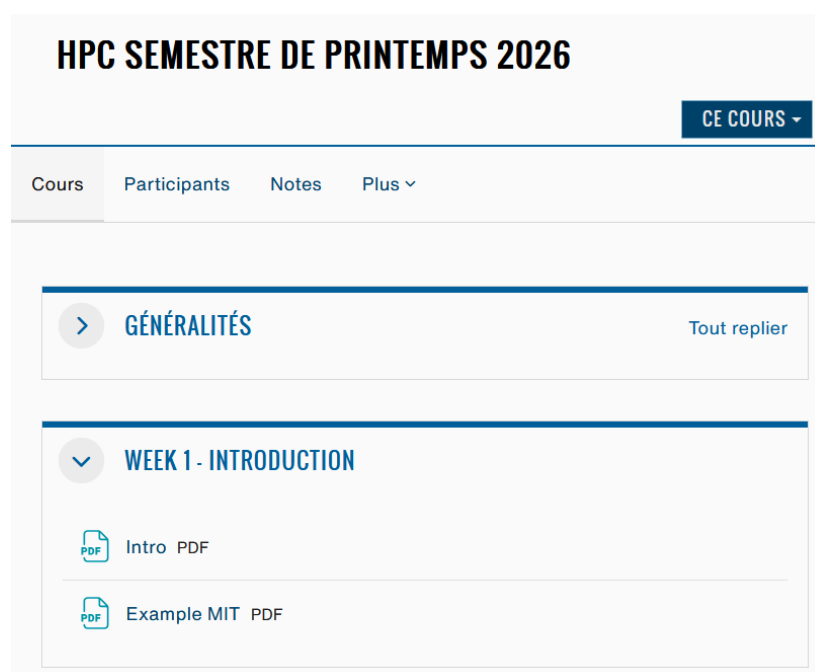


Figure 1: Moodle course

In many institutions, Moodle acts as a central platform for teaching activities, including document sharing, assessment management, and student tracking. For example, platforms like Cyberlearn (used within HES-SO) are built on top of Moodle and extend its capabilities to address institutional requirements.

As stated in the official Moodle [documentation](#) [2], one of Moodle's main strengths lies in its modular architecture. Rather than attempting to provide all functionalities required by educational institutions, Moodle focuses on being a robust Learning Management System while allowing interoperability with external systems offering complementary services.

“There are other types of software systems that are important for educational institutions [...] Generally, Moodle does not try to re-invent these areas of functionality. Instead, it tries to be the best LMS possible, and then interoperate gracefully with other systems that provide the other areas of functionality.”

This extensible architecture has led to the creation of a large ecosystem of plugins and integrations, allowing Moodle to be customized for various use cases. However, implementing advanced workflows often requires specific developments adapted to the platform’s architecture.

A well-known alternative to Moodle is Canvas [3]. While Moodle focuses on extensibility and customization, Canvas primarily emphasizes ease of use and straightforward setup.

The Moodle ecosystem is mainly developed using PHP [4], with JavaScript used for client-side functionalities. Therefore, the developed Moodle plugin must follow the same technological stack and adhere with the development guidelines defined by the Moodle platform.

3.2. GitLab

GitLab [5] is an open source Git repository management platform that provides tools for source code and project management and continuous integration/continuous deployment.

The main GitLab features include source code hosting and management through Git repositories, merge requests and code review workflows, issue tracking and project management tools, as well as CI/CD pipelines for automated testing and deployment. Additionally, GitLab provides extensibility capabilities through integrations based on Webhooks and REST/GraphQL APIs.

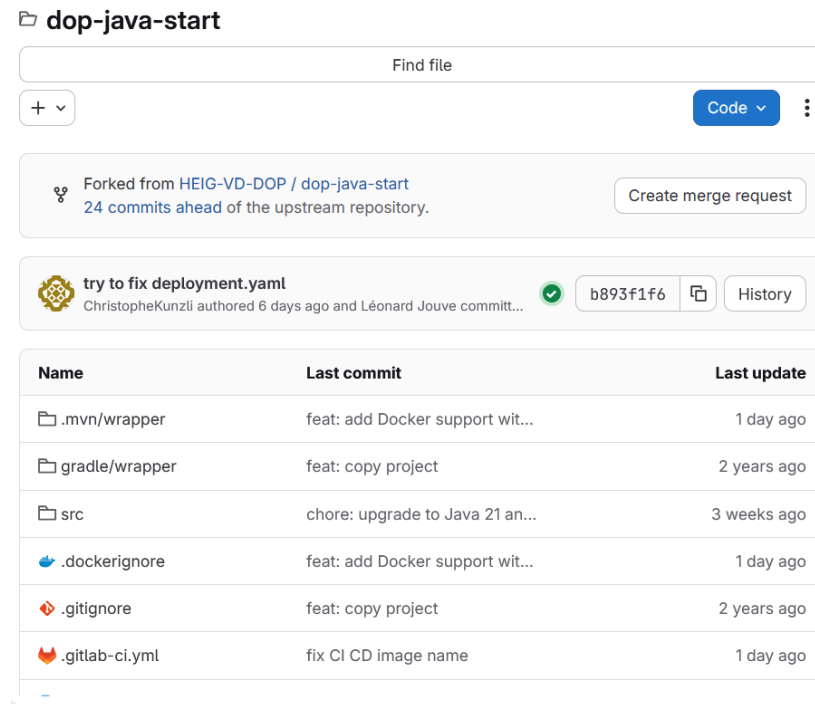


Figure 2: GitLab repository

GitLab is widely used in both industry and education due to its full DevOps life-cycle integration and strong automation capabilities. In academic contexts, it is commonly used for programming assignments, enabling students to collaborate and submit code while allowing teachers to review and evaluate contributions efficiently.

Another alternative is GitHub, a Git hosting platform that provides similar functionalities to GitLab. However, GitHub follows a different approach, relying more heavily on its developer community and ecosystem of external integrations, while GitLab provides a more unified platform for managing the software development lifecycle.

4. State of the art

There are 2 faces of this project:

- GitLab interactions through REST API
- Moodle plugin creation for the UI

We must link both to have a working integration.

4.1. GitLab Client

On the GitLab API side, there are multiple existing GitLab Client solutions. A HTTP Client is an abstraction layer wrapping REST API endpoints behind a user-friendly interface.

Several implementations exist across different programming languages and ecosystems:

- [GitLab Tools](#) [6] is a Java CLI
- [GitLab Haskell](#) [7] is a Haskell library
- [python-gitlab](#) [8] is a Python package

Since the Moodle plugin will be developed in PHP [GitLab PHP API Client](#) [9] seems to be the most suitable existing implementation. It provides support for the full GitLab API v4 specification.

However, Moodle plugins are designed to be self-contained and should avoid introducing unnecessary external dependencies. For this reason, a dedicated GitLab client will be implemented as part of this project, while taking inspiration from existing solutions and following their established design principles.

4.2. Moodle Plugin

Existing Moodle plugins provide various extensions to the platform, including additional activities, external integrations, and workflow automation. However, no widely adopted solution currently provides a complete integration between Moodle and Git repository hosting platforms.

This lack of an existing comprehensive solution motivates the development of a dedicated plugin capable of bridging Moodle with Git-based development platforms while following Moodle's plugin architecture and extension principles.

4.3. Integrations

Several existing solutions aim to bridge Learning Management Systems (LMS) with Git repository hosting platforms. These integrations mainly focus on simplifying programming assignment management, automating repository creation, and connecting educational workflows with software development tools.

4.3.1. GitHub Classroom

GitHub Classroom is an assignment management platform integrated with GitHub.

This solution automates the creation and setup of student repositories for teachers.

Teachers can specify an assignment name, due date, group size, starter code repository, CI jobs to run, a list of protected files to monitor for unauthorized modifications, and a feedback pull request workflow. Changes to the template repository can be synchronized across all student repositories through pull request. Reviewers are also notified when submissions are late.

GitHub Classroom is currently one of the most widely used solution in for computer science education assignments. However, its main limitation is that it is not directly integrated with the school LMS platform. Instead, it introduces an additional layer of complexity between the LMS and the Git hosting platform.

Another drawback is that it only integrates with GitHub, which is a closed-source platform. This can lead to vendor lock-in.

The workflows that teachers will be able to automate with our plugin will be heavily inspired by this solution, and all of its main features will be available.

On 22 May 2026, GitHub Classroom published a public announcement [10] the end of their service for 28 August 2026:

“For many educators using GitHub Classroom, you may know that this product has been operating under ‘maintenance mode’ for the past 18 months. We know this has been frustrating, and we want to make sure Classroom gets the investment it deserves.”

Following this decision, GitHub redirected users toward partner solutions:

- Codio [11]: A learning platform providing browser-based coding environments, automated grading capabilities, and LMS integration features. However, its LMS integration remains limited, and the platform is a closed-source premium solution.

- Classroom 50 [12]: A free and open-source assignment management platform developed by the Fifty Foundation. It enables instructors to distribute programming assignments, configure automated grading workflows through GitHub Actions, and manage student submissions using both web and command-line interfaces. This solution appears promising. However, it was only released on 1 July 2026, one week before this writing, and does not currently integrate directly with LMS platforms.

4.3.2. RepoBee

RepoBee [13] is a command-line tool that enables teachers to manage large numbers of student Git repositories across multiple version control systems. It features a powerful plugin system that allows users to either use existing plugins or develop their own. However, this solution does not integrate directly with any LMS platform.

4.3.3. Classmoji

Classmoji [14] is an open source LMS for teaching computer science. It is not widely adopted yet but is created with GitHub integration in the core. It claims to integrate GitHub Classroom with the LMS directly and add some additional features.

	Open source	LMS integra- tion	VCS indepen- dent	Extensible
GitHub Class- room	~	×	×	×
Codio	×	~	×	×
Classroom 50	✓	×	×	~
RepoBee	✓	×	✓	✓
Classmoji	✓	✓	~	~
This work	✓	✓	~	✓

Comparison of existing solutions

✓ Supported, ~ Partially supported, × Not supported.

As shown in the comparison, existing solutions either lack deep LMS integration, rely on proprietary platforms, or are limited to GitHub. Currently, there is no open-source, Moodle-native integration with GitLab that supports automated assignment workflows. This gap motivates the development of this project, which aims to provide an extensible, open-source solution for managing assignments through Version Control Systems directly within the LMS platform.

4.4. Academic work

4.4.1. Canvas GitLab Integration

Canvas GitLab Integration [15] is a research project aiming to enrich learning processes.

The paper highlights that separating version control systems (VCS) from learning management systems (LMS) leads to a fragmented and inefficient user experience, as students and instructors must switch between platforms. To address this, the authors propose a high-level programming framework that integrates Canvas and GitLab to reduce fragmentation and improve the overall educational experience.

It is a project in 3 parts:

- GitLab API
- Moodle API
- Moodle - GitLab integration API

All 3 projects are open source and written in Haskell.

The study investigated the experiences of Computer Science educators and students using Canvas and GitLab through a survey, identifying challenges caused by the lack of integration between the two platforms. The results showed that educators faced significant administrative workloads, including manually tracking student activity, linking GitLab projects with coursework, and monitoring student engagement. Students also reported difficulties associating coursework on Canvas with their GitLab projects and having to frequently switch between the two systems.

This report also mentions a key challenge in designing automated systems is balancing automation with flexibility. While automation can reduce repetitive tasks, improve efficiency, and minimize human error, excessive automation may limit users' ability to adapt workflows to specific needs or unexpected situations. Rigid automated processes can create frustration when users require control, or alternative approaches. Systems that provide too much flexibility, in the opposite may reduce the benefits of automation by increasing complexity and requiring additional manual effort. Therefore, effective system design should aim for an appropriate balance, automating labors tasks while preserving sufficient flexibility to allow users to make decisions and adjust processes.

5. Specifications

5.1. Objectives

This project aims to:

- Review the state of the art and identify the features and functionalities the project will include based on possibilities and limitations within the GitLab and Moodle APIs (for example: group management, project creation and feedbacks)
- Evaluate and identify most suitable architecture for the implementation
- The codebase should be modular, open to modifications and improvements
- Implement, develop and deploy the integration between Moodle and GitLab by following the DevOps principles
- Validate and test (optionally with representative users)

5.2. Deliverables

By the end of this work, we aim to have the following deliverables:

- Work report
 - Specifications document
 - Gantt chart
 - State of the art
- Work presentation
 - PowerPoint
 - Poster
- Plugin code published as open source
 - Usage documentation
 - Code tests
 - System architectural design
 - Demonstration of a working prototype

5.3. Expected result

The goal of this project is to have an extensible integration between GitLab (potentially any other Git repository) and Moodle. It would include an API automating all mentioned issues accessible from a Moodle integration plugin and thus improve teachers and students workflows.

The setup and use of the plugin should be easy and intuitive.

It should help teachers as well as students with their organization, documents / informations centralization and automate / simplify tedious workflows.

It should be released as open source to allow others to access, use, and build upon the work.

6. System architecture

TODO add more

The developed plugin(s) sit within the Moodle plugins layer and acts as the central bridge between the two platforms. It communicates with Moodle Core to read course data and create activity modules while a GitLab Client subcomponent handles all communication with GitLab. This client reaches GitLab REST API, through which it provisions and manages the GitLab resources to match the desired state. The REST API keeps both platforms fully decoupled: Moodle has no direct knowledge of GitLab internals, and GitLab requires no Moodle specific configuration.

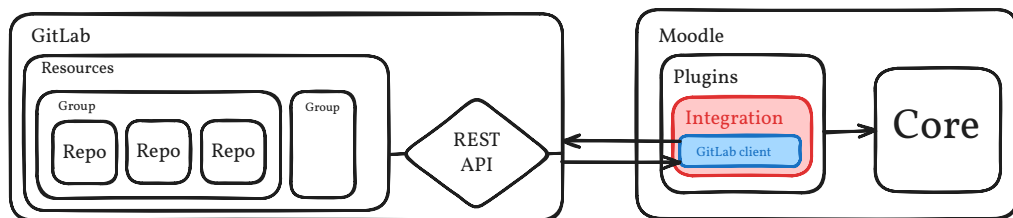


Figure 3: Moodle with GitLab integration

7. Features

The integration will be made with a Moodle plugin. The plugin will add an activity module^[16] named GitLab.

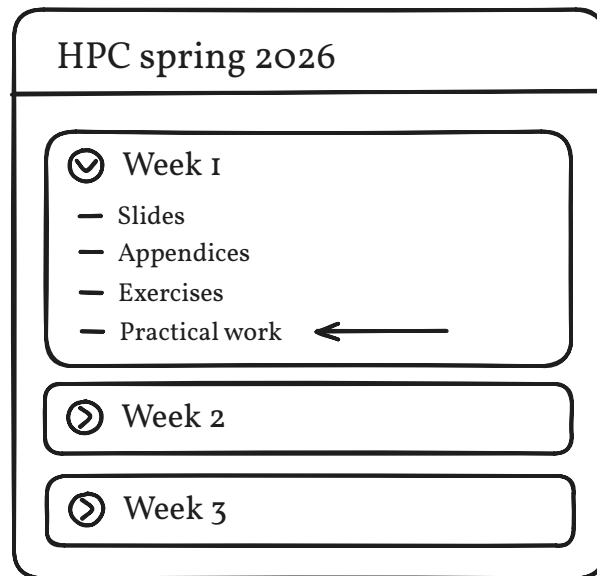


Figure 4: GitLab module on Moodle course page

7.1. Teachers

As a teacher, I want to:

- *define practical work properties such as name, description, instructions, due date, group size, reviewers*
- *create a GitLab activity module in Moodle so that I can manage a practical work linked to GitLab repositories.*
- *access a GitLab template repository so that I can prepare the initial project and solution.*
- *view all student groups so that I can monitor participation and project progress.*
- *create, edit, or remove student groups so that I can organize collaborative work.*
- *access each group repository so that I can review submissions efficiently.*
- *see the latest CI/CD pipeline results so that I can quickly identify failing projects.*
- *view individual contribution statistics so that I can assess each student's participation.*
- *download repositories source code so that I can review projects locally.*
- *know whether a group has already been graded so that I can avoid duplicate corrections.*

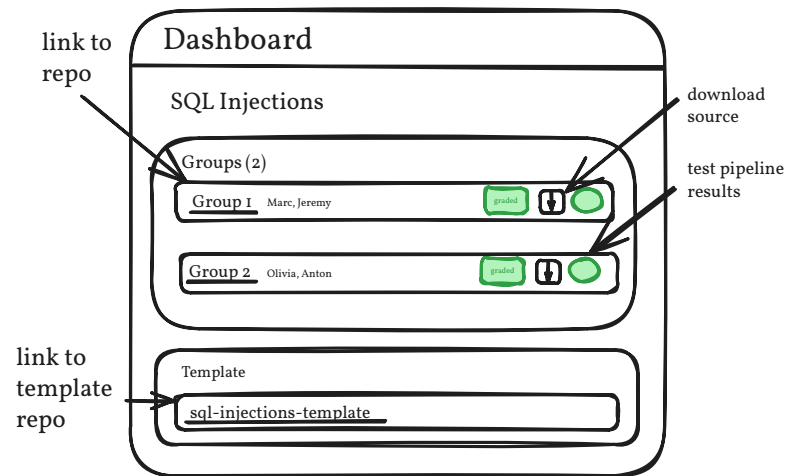


Figure 5: Teacher dashboard scaffold

7.2. Students

As a student, I want to:

- *view the list of existing groups so that I can join a team.*
- *join a group so that I can collaborate with other students.*
- *create a new group so that I can start a project team.*
- *access my group repository so that I can contribute to the practical work.*
- *view grades and feedback on Moodle and GitLab merge requests so that I can understand my evaluation.*
- *receive a notification when grading is completed so that I know when feedback is available.*
- *see the assignment due date in the Moodle calendar so that I can manage my schedule.*

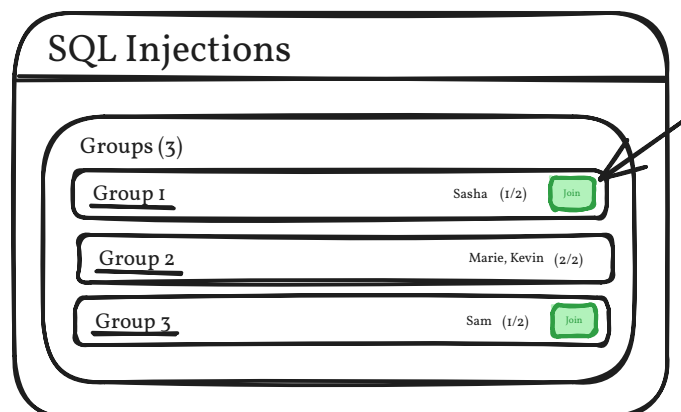


Figure 6: Student groups list scaffold

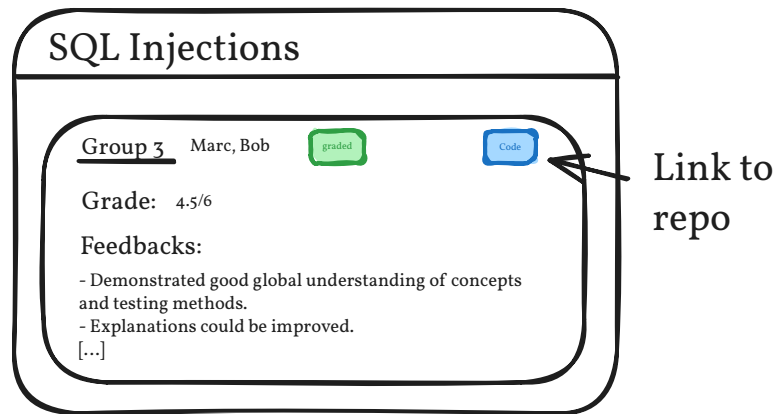


Figure 7: Student group scaffold

7.3. Automation

As a teacher, I want to:

- *create a GitLab group automatically when a Moodle activity is created so that setup is simplified.*
- *create a template repository automatically so that all student repositories share the same base project.*
- *see updates to the template repository propagate to student repositories through merge requests so that projects stay synchronized.*

As a student, I want to:

- *have a repository created automatically when my group is formed so that I can start working immediately.*
- *see the practical work instructions to be automatically added as GitLab issues so that I can easily track requirements.*
- *have a solution branch to become available after the deadline so that I can compare my work with the expected solution.*

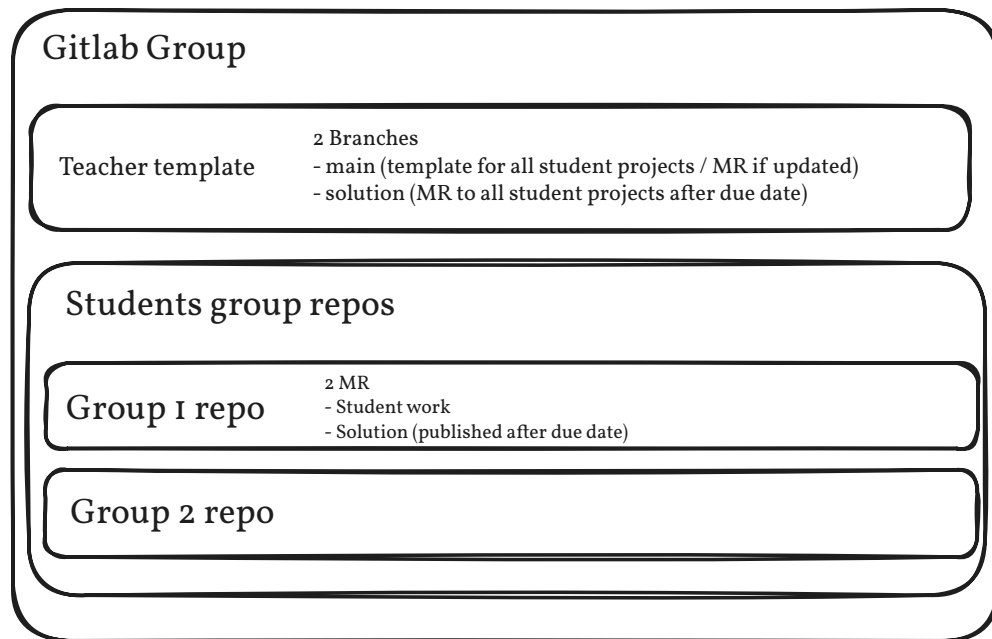


Figure 8: GitLab practical work structure

7.4. Other

Some custom integrations could be implemented with external tools, but these have been defined as out of scope for this project and could be added later through separate Moodle plugins.

Potential candidates include assessment platforms such as [Gradescope](#) [17] and [ANS](#) [18]. However, integrating with these proprietary platforms would introduce dependencies on third-party services and may not suit all teaching workflows, as instructors often have different preferences.

8. Integration Flows

TODO provide graphical representations of the interactions between the different components of the system

9. Development Process

The first months were mainly focused on establishing the requirements, the planning and the feasible features for the project.

I came out of this with a well defined feature list, Gantt chart Section 12.1 and Kanban board Section 12.4.

I also defined the development workflow to use on this project would be a Git feature branch strategy to ensure isolated and traceable changes. Each feature is implemented in a separate Git branch, allowing for review before integration into the main codebase. The task management is handled through a Kanban board [19]. Items are organized by priority and annotated with their dependencies. This ensures critical tasks are addressed first and visibility over blocking / critical tasks.

9.1. Proof Of Concept

I then realized as a proof of concept [20] a simple GitLab integration within Moodle. I created a Moodle module plugin. This kind of plugin adds a new type of resources a teacher can add to a course.

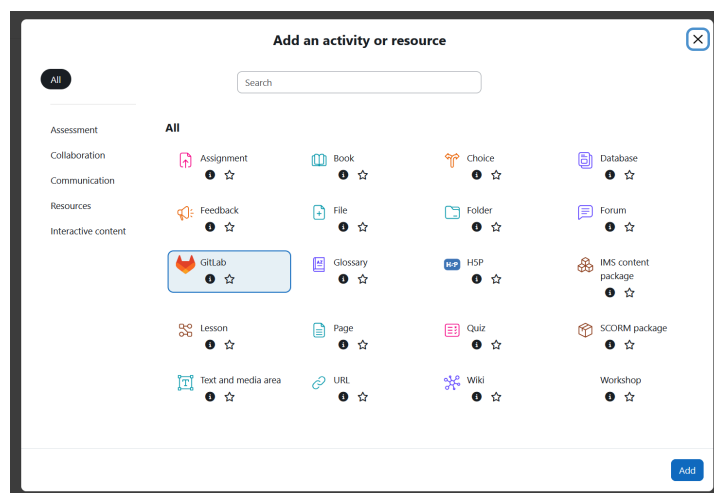


Figure 9: Modal for adding resources has a new **GitLab** type of module

While editing this resource, the teacher must define a GitLab API token. This will be used to perform API calls and create / configure the GitLab resources like repositories.

Once created, the students can open the module created and view a button which triggers the creating of a GitLab repository. They can also see a list of existing repositories in the GitLab group with a direct link to them.



Figure 10: GitLab module view



Figure 11: Resulting repository created on the selected GitLab group

I added Continuous Deployment [21] to provide a full featured reproducible development and testing environment.

I created an Infrastructure as Code using Terraform [22] allowing to create an AWS EC2 instance.

To setup the instance, I added Configuration as Code with Ansible [23] playbooks. It runs a Moodle instance and places it behind the Authentik [24] authenticity provider. It uses the Traefik [25] reverse proxy to manage communication between all applications and the ingress.

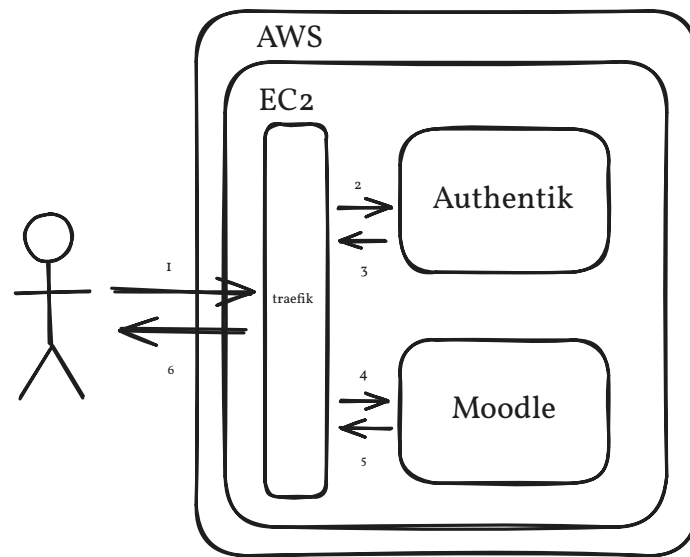


Figure 12: IaC architecture

I finally added a GitHub action to automate the updates after each push on the main branch of the project.

TODO outlines the main stages of the project's development and implementation process.

10. Solution

TODO details the final delivered solution.

11. Lessons Learned and Future Work

TODO summarizes the project's outcomes, discusses the challenges encountered and the lessons learned, and presents potential improvements and directions for future development.

12. Appendices

12.1. Planning

See [gantt.xlsx](#)

12.2. Weekly meeting reports

See [reports.pdf](#)

12.3. Source code

[GitHub Repository](#) [26]

12.4. Kanban Board

[Kanban Board](#) [19]

Bibliography

- [1] "Moodle." [Online]. Available: <https://moodle.org/>
- [2] "Moodle quote modular architecture." [Online]. Available: https://docs.moodle.org/dev/Moodle_architecture
- [3] "Canvas." [Online]. Available: <https://www.instructure.com/canvas>
- [4] "PHP." [Online]. Available: <https://www.php.net/>
- [5] "GitLab." [Online]. Available: <https://about.gitlab.com/>
- [6] "GitLab Tools." [Online]. Available: <https://gitlab.fhnw.ch/gitlab-tools/gitlab-tools>
- [7] "GitLab Haskell." [Online]. Available: <https://gitlab.com/robstewart57/gitlab-haskell>
- [8] "Python GitLab." [Online]. Available: <https://python-gitlab.readthedocs.io/>
- [9] "GitLab PHP API Client." [Online]. Available: <https://github.com/GitLabPHP/Client>
- [10] "GitHub Classroom transitioning to partners." [Online]. Available: <https://github.com/orgs/community/discussions/196615>
- [11] "Codio." [Online]. Available: <https://www.codio.com/>
- [12] "Classroom 50." [Online]. Available: <https://github.com/foundation50/classroom50/wiki>
- [13] "RepoBee." [Online]. Available: <https://repobee.org/>
- [14] "Classmoji." [Online]. Available: <https://github.com/classmoji/classmoji>
- [15] L. Schauer, R. Stewart, and M. Maarek, "Integrating Canvas and GitLab to Enrich Learning Processes." [Online]. Available: <https://doi.org/10.1145/3639474.3640056>
- [16] "Moodle activity module." [Online]. Available: <https://docs.moodle.org/501/en/Activities>
- [17] "Gradescope." [Online]. Available: <https://www.gradescope.com/>
- [18] "ANS." [Online]. Available: <https://www.ans.app/>
- [19] "Kanban board." [Online]. Available: <https://github.com/users/LeonardJouve/projects/3>

-
- [20] "POC." [Online]. Available: <https://github.com/LeonardJouve/moodle-gitlab-integration/pull/14>
 - [21] "CD." [Online]. Available: <https://github.com/LeonardJouve/moodle-gitlab-integration/pull/16>
 - [22] "Terraform." [Online]. Available: <https://developer.hashicorp.com/terraform>
 - [23] "Ansible." [Online]. Available: <https://docs.ansible.com/>
 - [24] "Authentik." [Online]. Available: <https://goauthentik.io/>
 - [25] "Traefik." [Online]. Available: <https://traefik.io/traefik>
 - [26] "Repository." [Online]. Available: <https://github.com/LeonardJouve/moodle-gitlab-integration>